R Programming for Data Analysis

Introduction to R

R is a programming language and environment built specifically for statistical computing and graphics. It was created by statisticians for statisticians, and remains one of the most widely used languages for data analysis, alongside Python, particularly in academia, biostatistics, and applied research.

R is typically used through an interactive console or an integrated development environment such as RStudio, which provides panes for scripts, plots, the console, and the workspace environment in a single window.

R Syntax and Basic Data Types

Vectors

The vector is the fundamental data structure in R. Even a single number is technically a vector of length one. Vectors are created with the c() function and support element-wise arithmetic.

```
x <- c(4, 8, 15, 16, 23, 42)
y <- x * 2
print(mean(x))
```

Data Frames

A data frame stores tabular data, where each column can hold a different data type, similar to a spreadsheet or a SQL table. Data frames are the primary structure used for statistical analysis in R.

```
df <- data.frame(
  name = c("Alice", "Bob", "Carla"),
  score = c(88, 92, 79)
)
summary(df$score)
```

Lists and Factors

A list can hold elements of different types and lengths, including other lists, making it useful for representing nested or heterogeneous data. A factor represents categorical data with a fixed set of possible levels, which R uses internally for grouping and modeling.

Control Flow and Functions

R supports standard control flow constructs such as if-else statements, for loops, and while loops, though vectorized operations are usually preferred over explicit loops for both speed and readability.

```
for (i in 1:5) {
  if (i %% 2 == 0) {
    print(paste(i, "is even"))
  }
}
```

Writing Functions

Functions are defined with the function keyword and can take default argument values. R functions return the value of the last evaluated expression by default, or an explicit value passed to return().

```
square <- function(x) {
  return(x^2)
}
sapply(1:5, square)
```

The Apply Family

Functions such as apply, sapply, lapply, and vapply apply a function across the elements of a vector, list, or matrix without writing an explicit loop, and are a hallmark of idiomatic R style.

Data Manipulation with dplyr

The tidyverse is a collection of packages that share a common design philosophy for data manipulation and visualization. dplyr, one of its core packages, provides a small set of verbs for the most common data manipulation tasks.

- filter() selects rows matching a condition
- select() picks specific columns
- mutate() creates new columns from existing ones
- arrange() sorts rows by one or more columns
- summarise() collapses many rows into summary statistics, often combined with group_by()

```
library(dplyr)

result <- df %>%
  filter(score > 80) %>%
  group_by(name) %>%
  summarise(avg_score = mean(score))
```

The pipe operator, written as %>% or the native |>, passes the result of one expression as the first argument to the next, allowing multi-step data transformations to be written as a readable left-to-right sequence.

Data Visualization with ggplot2

ggplot2 is the standard plotting package in R, based on the grammar of graphics. A plot is built by specifying a dataset, mapping variables to visual properties known as aesthetics, and adding geometric layers that determine how the data is drawn.

```
library(ggplot2)
ggplot(df, aes(x = name, y = score)) +
  geom_col(fill = "steelblue") +
  theme_minimal() +
  labs(title = "Scores by student", x = "", y = "Score")
```

Additional layers can add facets, which split a plot into a grid of subplots by a categorical variable, or statistical summaries such as a fitted regression line overlaid on a scatter plot.

Statistical Modeling in R

R was designed with statistical modeling as a first-class use case, and its formula syntax makes specifying models concise. A linear model is fit with the lm() function, using a formula of the form response ~ predictor.

```
model <- lm(score ~ hours_studied, data = df)
summary(model)
```

The summary output of a fitted model reports coefficient estimates, standard errors, t-values, and p-values for each predictor, along with overall model fit statistics such as R-squared and the F-statistic, making it straightforward to assess both individual predictors and the model as a whole.

Generalized Linear Models

The glm() function extends linear modeling to outcomes that are not normally distributed, such as binary outcomes modeled with logistic regression by specifying family = binomial, or count data modeled with a Poisson family.

R Markdown and Reproducible Reports

R Markdown combines narrative text, R code, and the output of that code, such as tables and plots, into a single document that can be rendered to HTML, PDF, or Word. This supports reproducible research, since the analysis and the writeup live in the same file and can be regenerated from the raw data at any time.

````
```{r}
summary(df)
```
````

Because code chunks execute when the document is rendered, R Markdown reports stay synchronized with the underlying data, eliminating the risk of a written report describing results that no longer match an updated dataset.